

Imperial College London

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Trajectory-Level Refactoring Analysis and Actionable Explanations Using IDE Traces

Author:
Ethan Hosier

Supervisor:
Dr Robert Chatley

Second Marker:

-

May 19, 2026

Abstract

Your abstract goes here

Acknowledgements

Contents

1	Introduction	6
1.1	Background	6
1.2	Motivation	6
1.3	Research Objectives	7
1.4	Contributions	7
1.5	Approach and Scope	8
1.6	Thesis Roadmap	8
2	Background and Related Work	9
2.1	Definitions and framing	9
2.1.1	Refactoring as behaviour-preserving transformation	9
2.1.2	Atomic refactorings, smells, and sequences	9
2.1.3	State-action-trace formalism	10
2.2	Outcome-based refactoring evaluation	10
2.2.1	Endpoint-centric quality improvement	10
2.2.2	What endpoint evaluation captures-and what it misses	10
2.3	Code smell detection and quality metrics	11
2.3.1	Smell detection as operationalising design issues	11
2.3.2	Quality metrics: structural and readability-oriented signals	11
2.3.3	Implications for process-quality metric construction	11
2.4	Refactoring detection and mining from code changes	13
2.4.1	From diffs to refactoring actions	13
2.4.2	Representation, coverage, and typical failure modes	13
2.4.3	Tie-in to this thesis	13
2.5	Refactoring recommendation and search/synthesis	14
2.5.1	Synthesis and path completion from partial edits	14
2.5.2	Search-based optimisation of refactoring sequences	14
2.5.3	Goal-directed navigation and architectural refactoring	15
2.5.4	Interactive and developer-in-the-loop recommendation	15
2.5.5	Beyond structural metrics: alternative objectives for ranking refactorings	16
2.5.6	Synthesis: what sequence-generation work enables, and what remains missing	16
2.5.7	Recent trajectory-formalism work in software engineering	16
2.6	Process-aware work and IDE trace capture	17
2.6.1	Logging developer interactions in IDEs	17
2.6.2	Granularity, segmentation, and semantic interpretation	17
2.7	Summary and positioning: toward process-quality evaluation	17
3	Methodology	19
3.1	Formal definitions and notation	19
3.2	The process score	19
3.3	Process-score weights	20
3.4	The cleanliness sub-score	22
3.5	The divergence-point detector	23
3.5.1	ORDERING detection	24
3.5.2	MANUAL_REFACTOR detection	24
3.5.3	REWORK detection	24
3.5.4	HYGIENE detection	24

3.6	Reorder synthesiser	24
3.7	Manual-refactor synthesiser	25
3.8	Rework synthesiser	26
3.9	Hygiene synthesisers	26
3.10	Validator and behaviour preservation	26
3.11	Rater design	27
4	Tool Architecture	28
4.1	Architecture at a glance	28
4.2	Event capture	29
4.3	Shadow repo reconstruction and worktree pool	29
4.4	Metrics calculation	29
4.5	Applying refactorings via Eclipse JDT	30
4.6	Synthesising counterfactual trajectories	30
4.7	Pipeline Phases	32
4.8	Dashboard	32
5	Project Plan and Timetable (Jan–Jun 2026)	33
6	Evaluation Plan	34
7	Ethical Issues and Data Handling Plan	35
A	First Appendix	36

List of Figures

1.1	Illustration of refactoring trajectories scored by process-quality metric J . The solid line shows the developer's observed trajectory, and the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.	8
4.1	High-level dataflow between the recording side (IDE plugin), the analysis pipeline, the embedded Eclipse JDT engine, and the dashboard. The shadow repo holds every reconstructed state – both user checkpoints and synthesised alternatives – as git commits in a per-session repository.	28

List of Tables

2.1	Candidate signals for constructing a process-quality objective. The thesis combines endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.	12
3.1	Notation used throughout the methodology chapter.	20
3.2	Production weights for Equation (3.1), the literature each weight rests on, and the magnitude recovery when only that term is active in the formula (full power-set ablation, $2^6 = 64$ variants per fixture; see Chapter 5).	21
3.3	The six sub-signals composing the cleanliness sub-score $C(s)$. All weights are uniform 1.0 following Laplace’s principle of insufficient reason (see composition rule below); the production weight bundle in the implementation reflects this.	22

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development: developers restructure code to improve readability, maintainability, and evolvability while keeping externally observable behaviour the same. Traditional definitions emphasise this preservation of behaviour and describe refactoring as a sequence of small, semantics-preserving transformations [1, 2, 3], and subsequent surveys categorise these refactoring operations and the tools that support them [4]. In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle when enhancing and maintaining code, and long-lived systems need continual adjustment (as argued in work building on Lehman’s laws and empirical studies of software maintenance). At the same time, refactoring can be costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated “refactoring-only” tasks [5, 6].

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common approach is to measure refactoring success as a change in static quality indicators between a “before” and “after” state: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related metrics to do with maintainability. Typical strands of work include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric changes to separate refactoring-like edits from functional ones, or to choose between different refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance by incorporating traceability alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers’ motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to determine developers’ motivations from commit artefacts). Alongside this, there is a substantial line of work that has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), which has enabled large-scale empirical studies of how often refactoring happens, what kinds occur, and in what contexts [7, 8].

However, these perspectives evaluate refactoring as a mapping from an initial program state to a final program state, leaving the *trajectory* between those states largely outside the picture. This matters because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes take detours, redo work, and often accept temporary degradation (e.g., in readability, modular structure, or test/build stability) before arriving at a final outcome. Two developers can arrive at the same end state via very different paths: one might take lots of disruptive steps, bounce between designs, or create avoidable churn, while another might get there via a steadier, more stable sequence. Existing endpoint-only evaluation cannot distinguish these different cases. Meanwhile, research on process realism suggests that refactoring is often “flossed” into ongoing development instead of being a separate dedicated task, which increases the chance of mixed-intent steps and noisy traces [5]. This makes the quality of the process (how a

developer navigates intermediate states) potentially important for both developer productivity and the health of the codebase as it evolves.

Concurrent work has begun to analyse refactoring trajectories under a state/action/observation formalism, qualitatively classifying failure modes of language-model agents on multi-file refactoring tasks [9] and characterising recurring action patterns in software-engineering agent traces [10]. A recent systematic literature review of LLM-based refactoring research likewise identifies process-aware evaluation as an open gap in the field [11]. As far as we are aware, however, no prior system both (i) scores a developer’s refactoring trace against an explicit process-quality metric and (ii) produces counterfactual reference trajectories tied to measurable divergence points. The closest work either synthesises a path to reach an end state (without judging the developer’s path) or recommends refactorings on an architectural target (without reference to an observed process). We instantiate the framing in a tool, evaluate it on a 45-session corpus of recorded refactoring sessions with hand-labelled ground truth, and report per-kind precision and recall alongside a sensitivity and ablation analysis of the underlying score formula.

1.3 Research Objectives

This project investigates the following thesis question: **given a known start state and a known end state, can we evaluate the quality of the developer’s refactoring process, and identify points where a better process was available?** The core idea is to explicitly model refactoring as a *process* over states, rather than judging it only by comparing endpoints. To do this, the project adopts a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits—including IDE refactorings and manual transformations—are treated as *actions* that produce a trace from the start state to the end state.

A core component will be a **process-quality metric** that scores traces according to criteria that will be defined and justified using existing research on code quality metrics, smell detection validity, and process-level signals. The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and “safety” proxies such as preserving build and test behaviour, reflecting the known limitations of relying on any one signal alone.

To go beyond scoring a single observed refactoring trace, the project will construct *counterfactual* alternatives in the form of other **reference trajectories** between the same start and end states which have been optimised to achieve a higher score under a chosen process-quality metric. This provides a reasonable basis for comparison: analysis can identify *divergence points* where the developer’s trajectory deviates from a higher-scoring path, and quantify how much of the gap in resulting refactoring quality is due to each divergence. This extends previous work that synthesises or recommends refactoring sequences: ReSynth synthesises a path to reach a target end state from a developer’s partial edits [12], Refactoring Navigator suggests refactoring steps to guide a system toward a desired target at an architectural level [13], and search-based refactoring treats refactoring sequences as an optimisation problem under explicit quality functions [14, 15]. While these systems show that refactoring sequences can be generated and optimised, they do not aim to assess the quality of a developer’s chosen process under an explicit process-quality objective. In this project, counterfactual generation supports *process evaluation*: it is used to construct a higher-scoring reference trajectory under the chosen metric and to explain key divergence points, rather than to automate the developer’s work or to find any arbitrary sequence of steps that can be followed to reach the end state.

1.4 Contributions

Concretely, this thesis makes the following contributions:

- A process-quality metric for refactoring trajectories, with each term grounded in cited prior work on code quality and developer process signals, and validated by a single-knob sensitivity sweep and a full power-set ablation analysis.
- A first-class divergence-point detector that classifies divergence into four kinds (ORDERING, MANUAL_REFACTOR, REWORK, HYGIENE) and produces a counterfactual reference trajectory for each.

- A 45-session hand-recorded corpus on a purpose-built Java fixture, with multi-label ground-truth annotations, together with the experimental drivers (sensitivity, ablation, divergence) used to evaluate the detector against it.
- An empirical evaluation of the detector that establishes per-kind precision and recall, surfaces what the score formula measurably captures, and identifies which categories of process improvement the formula is structurally insensitive to.

1.5 Approach and Scope

The approach proposed in this thesis targets refactoring as it actually occurs in real-world development, including sequences that involve both IDE-supported refactorings and manually performed edits. It initially focuses on refactorings in a single language (Java) to allow the use of mature parsing infrastructure, established quality metrics, and state-of-the-art refactoring mining (e.g., RefactoringMiner). A refactoring process is represented as a sequence of steps between code snapshots. Steps may correspond to commits or developer-defined checkpoints, and this representation can be refined using higher granularity IDE data where available.

The final system comprises (i) an IDE plugin for capturing refactoring traces and presenting feedback, and (ii) an analysis component that scores the observed refactoring process and finds higher-scoring alternative trajectories between the same start and end states under a chosen process-quality metric. Rather than attempting all possible refactoring operators in a given state, or calculating globally optimal refactoring trajectories, the goal is to produce a plausible reference trajectory which scores higher on the chosen metric, and to explain the points at which the developer’s trajectory diverged from it (e.g., where making a different earlier decision could have avoided churn, reduced temporary quality degradation, or preserved build/test stability).

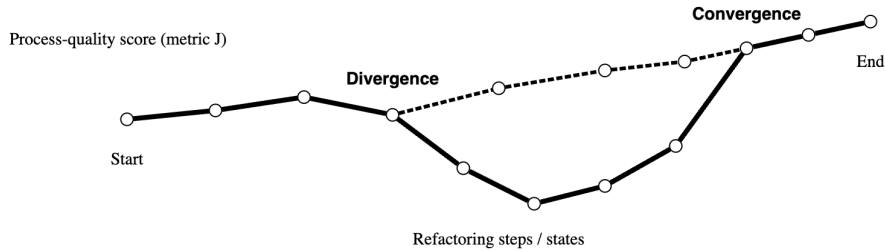


Figure 1.1: Illustration of refactoring trajectories scored by process-quality metric J . The solid line shows the developer’s observed trajectory, and the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.

1.6 Thesis Roadmap

The remainder of this thesis is structured as follows. Chapter 2 surveys related work in refactoring evaluation, mining, recommendation and synthesis, and positions the present work against that literature. Chapter 3 defines the process-quality metric and the divergence-point detector. Chapter 4 describes the tool’s architecture and implementation. Chapter 5 reports experimental results on the 45-session corpus and analyses threats to validity. Chapter 6 concludes and identifies directions for future work.

Chapter 2

Background and Related Work

This chapter reviews the background needed to evaluate *refactoring as a process* rather than just refactoring outcomes. It starts with established definitions of refactoring and the concept of refactoring sequences (§2.1). It then covers the more common outcome-based approach for evaluating refactoring (§2.2), before discussing the quality signals typically used to define “better code”, including code smells and static metrics (§2.3). It then looks at methods for detecting and mining refactorings from code changes (§2.4), which is what makes it possible to extract actions and traces from real development histories. The chapter goes on to survey refactoring recommendation and sequence synthesis (§2.5), prior work on capturing developer process traces in the IDE (§2.6), and concludes with the positioning of the present thesis against this body of work (§2.7).

2.1 Definitions and framing

2.1.1 Refactoring as behaviour-preserving transformation

Refactoring is defined as a change to a program’s internal structure intended to improve design qualities (e.g. readability, maintainability) while preserving externally observable behaviour. Most definitions emphasise that refactorings are applied as small, semantics-preserving steps and that safety is ensured through maintaining preconditions and/or validation (e.g. testing) [1, 2, 3]. Opdyke’s foundational work explains refactorings as program transformations equipped with applicability conditions (preconditions) intended to preserve behaviour under a static approximation [1]. Fowler’s catalogue helped popularise refactoring as an incremental activity performed by composing many small operations, grounded in developer-facing mechanics and motivations [2, 3]. Mens and Tourwé later survey the research landscape, bringing together definitions, taxonomies, and tool support, and emphasising the role of precondition checking and program analysis in automated refactoring [4].

In practice, the intent of maintaining behaviour during refactoring is often complicated by the realities of real-world development: refactorings are interleaved with feature and bug-fix work, may be partially performed, and may be mixed with behaviour-changing edits. Consequently, whether a change is “pure refactoring” is often ambiguous when viewed only through version history. This motivates treating behaviour preservation as an *intended property* of refactoring steps, while acknowledging that empirical traces may contain mixed-intent transitions.

2.1.2 Atomic refactorings, smells, and sequences

Research on refactoring often defines a set of *operators* (also described as atomic refactorings) such as EXTRACT METHOD, MOVE METHOD, or RENAME that can be composed together into larger transformations [2, 4]. These operators provide (i) a definitive set of actions for tools and (ii) a way to reason about refactoring sequences. The notion of *code smells* is also closely related: these are heuristic indicators that suggest potential design problems and thus potential refactoring opportunities [2, 3] in codebases. While smells are not formal criteria for correctness, they provide a widely-used bridge between developer intuition and measurable signals (discussed further in §2.3).

A key distinction of this thesis is between evaluating refactoring as an *endpoint mapping* (before vs after) and evaluating refactoring as a *trajectory* (a sequence of intermediate states). Even if

two trajectories share the same start and end states, they may differ substantially in intermediate degradation, churn, or disruption, which can be consequential for productivity and risk during the refactoring process.

2.1.3 State–action–trace formalism

To explicitly represent refactoring as a process, this thesis adopts a state/action/trace framing aligned with the view of refactoring as sequential operator application [2, 4]. Let a codebase snapshot be a *state* $s \in \mathcal{S}$ (e.g. the repository at a particular point in time). Let an *action* $a \in \mathcal{A}$ denote a developer-performed edit step, which may be an IDE-supported refactoring operation, a manually performed transformation, or a mix of the two. A refactoring process is then a *trace*

$$\tau = (s_0, a_0, s_1, a_1, \dots, a_{T-1}, s_T),$$

where each a_t transforms state s_t into s_{t+1} . Under this framing, “process quality” can be treated as a property of τ (and potentially its prefixes), rather than a property only of (s_0, s_T) .

Two practical issues arise from this. First, the granularity of steps (what constitutes one a_t) is not constant: steps may be commits, IDE events, or developer-defined checkpoints. Second, action labels may be partially observed or inferred (e.g. via refactoring detection tools). These motivate our review of refactoring mining (§2.4) and shape how we design process-quality metrics (§2.3).

This state/action/trace framing is consistent with recent software-engineering work that models agent and developer behaviour as a partially-observable Markov decision process, in which a trajectory is a sequence of (action, observation) pairs [9, 10]. We retain the explicit s_t notation in addition to actions because intermediate states—rather than only observations—are the primary object scored by the process-quality metric defined in the methodology chapter.

2.2 Outcome-based refactoring evaluation

2.2.1 Endpoint-centric quality improvement

A common way to evaluate refactoring is by comparing quality indicators computed on a “before” snapshot and an “after” snapshot. In this framing, refactoring is deemed successful if it improves a chosen set of quality proxies (e.g. reduced coupling, increased cohesion, reduced complexity, fewer code smells). This endpoint-focused perspective appears in both empirical studies of refactoring impact and in automated recommendation systems that rank refactoring candidates by their predicted improvement.

A representative example is *search-based refactoring*, which frames refactoring as an optimisation problem over sequences or sets of refactoring operations. Harman and Tratt demonstrate multi-objective search at the design level, explicitly treating coupling and cohesion as competing objectives and producing Pareto-optimal trade-offs [14]. Later work extends this to many-objective settings and introduces constraints or preferences that reflect practical concerns such as test coverage or prioritisation [15]. These approaches make it explicit that “better” is inherently multi-dimensional, and collapsing everything into a single quality score can be overly restrictive.

This same outcome-based ranking also appears in recommender systems that evaluate alternative refactoring solutions. For example, some systems extend beyond structure-based metrics by incorporating additional information such as requirements traceability, arguing that designs can be “better” not only in properties derivable from the code structure, but also in maintainability tasks like impact analysis [16]. More recently, researchers have revisited the problem of selecting among large sets of refactoring candidates and investigated which code characteristics tend to correlate with positive or negative impacts in practice [17].

2.2.2 What endpoint evaluation captures—and what it misses

Endpoint-based evaluation is best-suited to answer questions of the form: “is the end state better than the start state under metric Q ?” It provides a simple basis for automated recommendation (simulate each candidate and then choose the one that maximises ΔQ) and for empirical analysis (associate refactoring events with changes in quality proxies). It also aligns with the fact that many quality metrics are defined as functions of a single codebase snapshot.

The downside is that it ignores the refactoring process itself, and therefore cannot distinguish between trajectories that reach the same endpoint. In real refactoring, intermediate states can temporarily degrade in ways that matter: code may become harder to understand, modular structure may be disrupted, or build/test stability may regress before it improves again. Furthermore, the *cost* of reaching an improved endpoint is not solely captured by the endpoint metrics alone: disruptive changes can increase churn, complicate review, and introduce integration risk even if final static metrics improve. Empirical work on modularity improvement highlights this trade-off between optimisation and disruption, showing that metric gains can come with substantial structural disturbance [18]. This motivates evaluating not only final quality, but also the *trajectory cost* and stability of the path taken.

These limitations are central to the thesis: the goal is not to replace outcome-based quality metrics, but to use them within a process-level evaluation that can treat intermediate degradation, rework, churn, and stability as first-class terms in a process quality objective.

2.3 Code smell detection and quality metrics

2.3.1 Smell detection as operationalising design issues

Code smells are a common way to link between qualitative design concerns and measurable indicators [2]. Smell detectors typically detect these smells via rules or learned models. They usually rely on structural properties (e.g. size, coupling, cohesion) and, increasingly, lexical and semantic features also. A prominent rule-based system is DECOR, which defines smells and design anti-patterns using a set of measurable symptoms and corresponding detection rules [19]. DECOR is widely used in empirical studies because its definitions are explicit and it can be executed efficiently at scale.

Despite being useful, smell detectors are not ground truth. Comparative studies often show disagreement between tools in both precision and recall, and that performance is highly variable across different systems and smell types [20]. These differences are largely derived from how each tool defines smells in practice: thresholds, feature sets, and implementation details. Furthermore, detectors may produce false positives: code sections flagged as “smelly” when there is no clear negative impact or that reflect acceptable design trade-offs [21]. For process evaluation, these issues imply that smell counts should be treated as one signal among several rather than as a sole objective: combined with complementary indicators (structural metrics, readability proxies, and process-level safety signals), they contribute useful information without carrying the result alone. The process-quality metric defined in the methodology chapter adopts this combinatorial approach, treating smell-based terms as one component of a multi-term score rather than the dominant signal.

2.3.2 Quality metrics: structural and readability-oriented signals

Static metrics offer another way to quantify design quality at the granularity of classes, methods, and packages. Classic metric families quantify coupling, cohesion, complexity, and size, and are widely used as proxies for maintainability. Recent work has also argued for using readability-oriented signals that capture aspects not well-modelled by purely structural measures. For example, readability models that combine structural features (e.g. line structure) with textual features (such as identifier patterns) tend to align better with human judgements of readability [22]. These signals are particularly relevant for evaluating refactoring because readability and understandability are common motivations for refactoring and are often affected by intermediate states.

2.3.3 Implications for process-quality metric construction

The process-quality metric $J(\tau)$ used in this thesis will draw on these signals, but it must also account for their limitations. The literature points to three key design principles:

1. **Use multiple signals.** No single metric or smell detector provides reliable ground truth across contexts [20, 21]. Combining complementary signals can reduce sensitivity to tool-specific noise.

2. **Differentiate endpoint improvement from trajectory cost.** Outcome-based improvements remain valuable, but disruption and intermediate degradation can be significant even when the final snapshot improves [18].
3. **Prefer interpretable components.** For a process critique to be actionable, the metric should be decomposable into terms that can be explained (e.g., readability dip, churn spike, smell increase), rather than a single opaque score.

A further practical implication is that process evaluation should include an explicit notion of developer **effort**, such as the length of the trajectory (number of steps) and/or the magnitude of edits required between successive states. While fewer steps are not inherently better (e.g., small “micro-steps” may reduce risk), step count and edit effort provide a natural regulariser against unnecessary detours and rework, complementing quality-improvement terms and measures of disruption. Cost terms like these are commonly treated as optimisation objectives alongside structural quality goals in search-based refactoring and recommendation settings (e.g., minimising change while improving quality), and are therefore suitable candidates for trajectory-level scoring [14, 15].

Table 2.1 summarises candidate signals, their typical benefits and limitations, and examples of available tooling in the Java ecosystem. The purpose of the table is not to fully specify the final form of the process-quality metric $J(\tau)$ (e.g., precise term weights and normalisation), but to motivate the choice of component signals and the design principles that the final metric will comprise. A precise definition of $J(\tau)$ is given in the methodology chapter and used consistently throughout the thesis evaluation.

Signal	Pros	Cons / threats	Typical tooling
Structural metrics (coupling/cohesion/complexity/size)	Widely used; efficient; interpretable at class/method granularity	Proxy validity depends on context; may incentivise disruptive restructuring; metric interactions	Static analysis metric extractors; IDE/CI integrations
Smell counts / severity (rule-based)	Directly targets design issues; aligns with developer’s motivations	Detector disagreement and false positives; threshold sensitivity; context dependence	DECOR-style detectors [19]; other smell tools [20]
Readability models	Closer to human comprehension than structural-only metrics; captures lexical/style clues	Model generalisation and dataset bias; may be language/style dependent	Readability predictors [22]
Churn / disruption (LOC changed, file moves/renames)	Captures cost and instability of change; naturally trajectory-oriented	Churn may be necessary for large refactors; can penalise required restructuring	VCS mining; diff statistics
Process length / effort (number of steps; edit magnitude)	Trajectory-native cost signal; discourages detours/rework; easy to compute	Sensitive to step granularity (commits vs. checkpoints vs. IDE events); may penalise safe micro-steps; requires calibration	VCS checkpoints; IDE logs; diff/AST edit size
Build/test preservation proxies	Direct safety signal; aligns with “behaviour-preserving intent”	Requires runnable builds/tests; noisy due to flaky tests; environment dependence	CI logs; local build/test runs
Traceability-augmented signals	Reflects maintenance tasks beyond code structure; complements metric-only ranking	Requires trace links (often unavailable); domain-specific assumptions	Traceability-based ranking methods [16]

Table 2.1: Candidate signals for constructing a process-quality objective. The thesis combines endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.

2.4 Refactoring detection and mining from code changes

2.4.1 From diffs to refactoring actions

To evaluate a process trace in practice, one usually needs to either (i) capture developer actions directly (for example, IDE event logs) or (ii) infer actions from the differences between snapshots (like commits). Much of this work focuses on the second case: given two versions of a program, infer whether a refactoring happened and, if it did, identify which refactoring types occurred.

A key technical enabler is *structured code differencing*, usually built on abstract syntax trees (ASTs). This makes it possible to detect moves, renames, and other non-local edits that line-based diffs tend to miss. GumTree is a representative AST differencing approach: it builds fine-grained mappings between AST nodes and produces edit scripts that line up better with the structural changes developers actually intended [8]. This kind of differencing is often used as a base layer for refactoring detection.

Building on these foundations, modern refactoring miners detect refactoring instances by combining structural mappings with refactoring-specific heuristics. RefactoringMiner is a well-known tool that detects a wide range of refactoring types by analysing successive revisions, using structured matching to relate program entities across versions [7]. RefDiff takes a similar direction: it detects common refactoring types in version histories using similarity heuristics and static analysis [23]. Together, these tools support large-scale mining studies and can also provide an *action vocabulary* for traces, for example by labelling a transition $s_t \rightarrow s_{t+1}$ with a multiset of detected refactoring operations.

2.4.2 Representation, coverage, and typical failure modes

Refactoring detection tools typically output refactoring instances using a taxonomy of operator types (e.g., MOVE METHOD, EXTRACT CLASS) and the program entities involved. This is useful because it matches the operator-based view of refactoring used in catalogues and surveys [2, 4]. However, the literature repeatedly reports several limitations, and these are especially important when doing process evaluation.

Mixed and tangled changes. Version control commits often mix refactoring and functional edits, or they bundle multiple unrelated activities together. In these cases, a commit-level transition can include both behaviour-preserving and behaviour-changing edits, and refactoring detection may only return partial results or it may be ambiguous. This reduces how interpretable the actions extracted from snapshots are, and it motivates careful trace construction (for example, using finer checkpoints) and methods that are robust to noisy labels.

Incomplete coverage and composition effects. Detectors only cover a finite set of operators, and refactorings can also be composed (e.g., move+rename) in ways that make matching harder. Some refactorings are done manually without tool support, and they might not fit the patterns that detectors expect. For process evaluation, this means that “actions” should be treated as only partially observed: detected refactorings are useful as informative features, but they are not an exhaustive description of developer intent.

Behaviour preservation is not verified. Detection is usually structural: it infers that a change *looks like* a refactoring, but it does not prove that behaviour was preserved. Because of this, downstream evaluation should not treat “detected refactoring” as the same thing as a “safe refactoring”. It should also use independent safety proxies where possible (for example, whether the build passes or tests still pass).

2.4.3 Tie-in to this thesis

Refactoring mining is important infrastructure for building traces and for characterising actions at scale [7, 23]. However, the contribution of this thesis is not to improve refactoring detection accuracy. Instead, mined refactorings are used as part of the observational layer: they provide labels and features that describe transitions between states, which can support constructing a process-quality metric and generating reference trajectories. The next part of this background builds

on these ideas to discuss prior work on refactoring sequence generation and navigation, which is directly relevant to constructing higher-scoring reference trajectories under a chosen objective.

2.5 Refactoring recommendation and search/synthesis

The previous sections motivate two requirements for process-level refactoring evaluation: (i) a way to represent and score a developer trace, and (ii) a way to construct counterfactual *alternative* trajectories for comparison. A substantial body of work already generates refactoring suggestions and sequences, but it usually targets objectives that are different from evaluating a developer’s process. Broadly, existing approaches fall into three overlapping families: (a) *synthesis* approaches that reconstruct or complete refactoring sequences from examples or partial edits; (b) *search-based* approaches that optimise refactoring sequences under quality objectives; and (c) *interactive* approaches that adapt recommendations based on developer feedback. This section reviews these families and explains how they can support, but do not directly solve, process-quality evaluation.

2.5.1 Synthesis and path completion from partial edits

One way to generate counterfactual alternatives is to treat refactoring as a program transformation problem: given a start program and a target (or partially edited) program, synthesise a sequence of refactoring operators that carries out the transformation. ReSynth is a representative system in this space [12]: it synthesises a sequence of IDE-supported refactorings consistent with a developer’s partial edits, constraining the search by the entities involved in those edits and by the operators’ preconditions. Its objective is *reachability and consistency* (does some operator sequence reproduce the developer’s edits?) rather than process-quality evaluation. The general lesson for this thesis is that refactoring can be framed as a structured search problem over refactoring operators, and that the search space can be constrained using evidence from the developer trace itself rather than enumerated naively.

A closely related line of work generalises “from examples” synthesis beyond classical refactorings. REFAZER learns program transformations from input-output examples of code edits [24]. Instead of assuming a fixed refactoring catalogue, it synthesises AST-level rewrite rules from a small number of examples and can apply them to automate repetitive edits. While REFAZER is not a refactoring miner in the narrow Fowler sense, it is relevant to counterfactual generation in two ways: (i) it offers a way to infer candidate transformations without predefining an exhaustive operator set; and (ii) it shows that “plausible alternatives” can be derived from edit evidence, which may help bridge the gap between IDE-supported refactorings and manually performed edits.

Relevance and limitations for process evaluation. Synthesis systems mainly aim to *reach* a target program (or a class of targets that are consistent with partial edits) and to do so using sequences that satisfy refactoring preconditions [12]. The objective is reachability and consistency, not evaluating the quality of a developer’s chosen trajectory. In particular, synthesis does not completely answer whether the developer’s intermediate decisions were efficient, low-churn, or risk-minimising under a chosen process objective. Still, synthesis is useful as a source of *candidate alternative trajectories*: it provides a principled way to generate sequences that are likely to be feasible and realistic in a development environment.

2.5.2 Search-based optimisation of refactoring sequences

A second family treats refactoring recommendation as an optimisation problem. Search-based refactoring methods explore sequences or sets of refactorings and then select those that optimise one or more objective functions based on quality metrics [14]. Harman and Tratt argue explicitly for multi-objective formulations (e.g., coupling vs. cohesion) and produce Pareto fronts of trade-offs, rather than forcing design quality into a single scalar [14]. These methods are appealing because they make “better” more measurable, and they can produce many candidate solutions that show the trade-offs more clearly.

Later work extends these formulations to more realistic settings by adding extra objectives. For example, many-objective approaches include not only quality improvements, but also concerns like prioritisation and how refactoring effort is spread across the codebase [15]. These objectives are close to process evaluation because they start to encode costs and constraints that show up

during development (e.g., avoiding refactoring that is too concentrated, or respecting priorities), instead of treating refactoring as only a structural optimisation.

Search-based techniques also appear in work that looks at the tension between metric improvement and disruption. Empirical studies of modularity optimisation show that maximising cohesion/coupling improvements can cause substantial disruption compared to the original modular structure [18]. This line of work continues into the present: recent many-objective formulations explicitly treat *architectural distance*—a direct disruption proxy—as a first-class Pareto objective alongside structural quality [25], and related work has extended the framework with socio-technical cost proxies such as reviewer availability [26]. This matters for process evaluation: even if an end-point configuration is “better” under structural metrics, the path to reach it can be very disruptive, and disruption itself may be a cost term that process evaluation should capture.

Relevance and limitations for process evaluation. Search-based refactoring provides two key ingredients: an explicit objective function and a mechanism to generate candidate sequences. However, most work evaluates the *recommended* refactoring sequence, not a human trace. The algorithm’s output is treated as the solution, and success is measured by the improvement it produces (or by user study preference for the recommendation). This is different from the present thesis, where the goal is to evaluate a developer’s observed trajectory *relative to* alternative trajectories under an objective that is meant to capture process quality. Still, search-based methods give a natural way to construct a *reference trajectory*: given a start state (and potentially an end constraint), they can produce a plausible trajectory that scores highly under the chosen process objective and can act as a baseline for comparison.

2.5.3 Goal-directed navigation and architectural refactoring

A third family focuses on guiding refactoring toward a target design. Refactoring Navigator is one of these: it takes a current system and a desired target architecture/design, then recommends a sequence of refactoring steps using a search-based strategy, presented through a navigation metaphor [13]. The key idea is to guide developers step-by-step toward a goal, rather than just ranking isolated refactorings. Controlled evaluations report large reductions in task time and manual edits compared to unguided refactoring [13], which suggests that sequence-level guidance can change how developers actually refactor.

Goal-directed refactoring is particularly relevant for the “grand scheme” view of this thesis: it is natural to compare an observed developer trajectory to a higher-scoring “reference” trajectory that reaches a comparable end state. Navigation systems show that it is feasible to produce these stepwise trajectories, and to map high-level goals down to low-level refactoring actions.

Relevance and limitations for process evaluation. Navigation systems typically optimise toward reaching a target design, and they may use design metrics to guide progress [13]. They are not built to evaluate a human’s process quality, and they do not try to explain where a human trace diverged from a better trajectory. However, they provide strong evidence that (i) refactoring can be treated as a sequential decision problem, and (ii) sequence-level suggestions can be computed and presented in ways that developers can actually use.

2.5.4 Interactive and developer-in-the-loop recommendation

Purely automatic optimisation can end up misaligned with developer intent, constraints, or context. Interactive refactoring recommenders address this by incorporating user feedback during the search process. Alizadeh and Kessentini propose an interactive and dynamic search-based approach that presents refactoring recommendations iteratively and updates the search based on developer decisions [27]. This highlights that refactoring recommendation is not only about finding a high-scoring sequence under fixed metrics; it also involves keeping recommendations aligned with developer preferences and constraints as they evolve during a task.

Relevance and limitations for process evaluation. Interactive recommenders are not evaluation tools, but they are closely aligned with the instrumentation and delivery mechanism envisioned by this thesis. If process evaluation is delivered in an IDE, the feedback has to be understandable, minimally disruptive, and sensitive to developer context. Work on interactive recommendation

provides ideas for presenting sequence-level suggestions and for building feedback loops into the developer’s workflow [27]. The remaining open problem is to shift from “recommend to optimise” to “compare and evaluate an observed trajectory relative to alternative, counterfactual trajectories”.

2.5.5 Beyond structural metrics: alternative objectives for ranking refactorings

A recurring theme in recommendation research is that structural metrics alone may not be enough to capture what makes a refactoring “good”. One example is ranking refactoring candidates using requirements traceability alongside code metrics. Approaches that include traceability treat “quality” as partly determined by how well requirements map to code, and they are motivated by maintenance tasks like impact analysis and comprehension [16]. This supports the idea that process-quality objectives should not be assumed to be purely structural: the relevant costs and benefits of refactoring can include non-structural properties and context-specific constraints.

Recent work also looks at developer motivations for refactoring, and tries to align recommendations with what developers are likely to prioritise. Studies that mine refactoring operations and repository history investigate product/process signals that correlate with refactoring activity and with developer intent, including proposals that use LLMs to infer motivations from commit artefacts [28, 29]. While these works mostly focus on *when* and *why* refactoring happens, rather than *how well* it is carried out as a process, they suggest that developer-aligned objectives may need to incorporate process signals and contextual information.

2.5.6 Synthesis: what sequence-generation work enables, and what remains missing

Sequence generation and recommendation systems establish that it is feasible to:

- generate refactoring sequences that realise a transformation or reach a target (synthesis/navigation) [12, 13],
- optimise sequences under explicit metric objectives (search-based refactoring) [14, 15],
- incorporate developer feedback into iterative suggestion mechanisms (interactive recommendation) [27],
- and broaden “quality” beyond structural metrics where additional artefacts exist (e.g., traceability) [16].

However, these systems do not directly answer the two central questions posed by process-level evaluation:

1. *Was the developer’s observed trajectory good under an explicit process-quality criterion?*
2. *Where did the developer’s trajectory diverge from a higher-quality trajectory, and can that divergence be explained?*

The distinction is subtle but important. Recommendation and synthesis work typically treats the algorithm’s output as the primary object and evaluates its outcome or usefulness. Process evaluation instead treats the *developer trace* as the primary object and seeks to compare it against counterfactual trajectories constructed under a process-quality objective.

2.5.7 Recent trajectory-formalism work in software engineering

Recent work has begun to formalise software-engineering agent behaviour as a trajectory of (action, observation) pairs in a POMDP-shaped environment [9, 10, 30], and a recent systematic literature review of LLM-based refactoring research identifies process-aware evaluation as an open gap in the field [11]. These developments confirm that the trajectory framing is gaining traction in concurrent software-engineering research, but the work to date targets LM-agent behaviour and benchmark task success rather than human developer process quality. Empirical observational studies independently confirm that real-world refactoring proceeds as ordered sequences of operations interleaved with other change activities [31], motivating the trajectory-level framing adopted in this thesis.

2.6 Process-aware work and IDE trace capture

Evaluating a refactoring process requires access to traces: sequences of intermediate states and actions. There are two broad sources of traces: (i) version control histories (commits/checkpoints), and (ii) IDE-level interaction logs that capture finer-grained behaviour. This section focuses on IDE logging and process-aware studies, since they show it is feasible to capture refactoring traces at a useful granularity, and they also highlight practical limitations that end up determining evaluation design.

2.6.1 Logging developer interactions in IDEs

Multiple studies have collected large-scale logs of developer interactions to mine workflows and behavioural patterns. Damevski et al. present an approach for mining sequences of developer interactions in Visual Studio using telemetry-like event streams, with the aim of discovering frequent patterns of usage without necessarily recording the full code content [32]. This is relevant for trace collection because it shows that useful traces can be captured as structured events (e.g., commands, navigation actions, edits) while still maintaining the developer’s privacy.

Poncin et al. use Eclipse plugins to collect detailed event logs and then apply process mining techniques to infer developer workflows [33]. This gives a clear example of turning low-level interaction events into higher-level activity models, such as sequences of edits, navigation, compilation, and tool invocations. It also shows some of the practical challenges: event streams are noisy, developers multitask, and mapping raw events to semantically meaningful steps needs careful segmentation and interpretation.

More recent infrastructure work has pivoted toward capturing developer-LLM interactions rather than refactoring-specific developer process traces. *CodeWatcher* is a representative example: a lightweight client-server VS Code extension that captures fine-grained, semantically labelled IDE events (LLM-tool insertions, deletions, copy-paste, focus shifts) with timestamps to support post-hoc reconstruction of coding sessions [34]. While the underlying telemetry mechanisms are directly applicable to refactoring traces, the 2023–2025 literature on IDE-side activity mining for refactoring specifically has been quiet—the broader process-mining-in-SE community has migrated toward repository and CI/CD logs rather than IDE event streams, and recent activity-recognition work targets LLM-coding patterns rather than refactoring sequences. Classical IDE-side activity mining for refactoring trajectories therefore remains an area where canonical references are over seven years old.

2.6.2 Granularity, segmentation, and semantic interpretation

IDE logging raises the question of what counts as a “step” in a refactoring process. At commit granularity, steps are coarse and often tangled with non-refactoring changes. At event granularity, steps are extremely fine and need to be aggregated into coherent actions. The process mining and interaction analysis literature highlights that segmenting activity into tasks/episodes is not trivial [33, 32]. For refactoring in particular, a single conceptual refactoring can involve several micro-actions (navigate, edit, run a refactoring tool, adjust code, run tests). On the other hand, a short burst of edits might correspond to multiple actions.

A process-evaluation system therefore has to choose a step definition that is both (i) practical to extract reliably and (ii) meaningful enough to allow for interpretation.

2.7 Summary and positioning: toward process-quality evaluation

The literature discussed in §2.1–§2.6 establishes a strong foundation for modelling and measuring refactoring, but it also leaves open a clear space for process-level evaluation.

First, refactoring is well-defined as behaviour-preserving and supported by catalogues of operators and by tool-based precondition checking [1, 2, 4]. Second, most evaluation work is still focused primarily on endpoints: it assesses refactoring success by looking at changes in quality indicators between start and end snapshots, including structural metrics and smell-based proxies [14, 19]. These signals are useful, but they are imperfect and can disagree across tools, which motivates combining multiple signals and metrics [20, 21, 22]. Third, refactoring mining and AST

differencing provide practical ways to infer refactoring actions from code changes, which enables trace reconstruction from repository histories [7, 8]. Fourth, recommendation and synthesis systems show that it is feasible to generate refactoring sequences and plan toward final goal states [12, 13, 27], and that “quality” may need to include contextual objectives beyond just structural metrics [16, 28, 29]. Finally, IDE logging research shows that fine-grained traces can be captured and analysed, but it also highlights challenges around the segmentation and interpretation of them [33, 32, 35].

While there has been substantial research on *detecting* refactorings, *recommending* refactorings, and *synthesising* refactoring sequences, as far as we are aware no prior system both (i) scores a developer’s refactoring trace against an explicit process-quality metric and (ii) produces counterfactual reference trajectories tied to measurable divergence points. The closest existing work—sequence synthesis, search-based recommendation, and the recent trajectory-formalism studies surveyed above—generates or analyses sequences as the primary object, treating the algorithm’s output as the solution rather than evaluating a developer’s chosen path against alternatives. Existing research generally does not define and evaluate a criterion for “good refactoring process” that penalises intermediate degradation, disruption, or instability, nor does it provide a specific method to compare a developer’s refactoring trajectory against alternative, higher-quality trajectories (relative to the chosen metric) and to explain divergence points from those trajectories.

This thesis addresses this gap by treating refactoring as a trajectory optimisation and comparison problem: given a start state, an end state (or end constraint), and an observed trace, defining a process-quality objective and constructing a higher-scoring reference trajectory for comparison. This leads to the overall thesis question: *given a known start state and end state, can we produce actionable explanations of a developer’s refactoring process by constructing a plausible reference trajectory and highlighting where and why the developer’s path diverged?*

Chapter 3

Methodology

This chapter defines the three main contributions of this thesis: the process-quality score, the divergence-point detector, and the per-kind counterfactual synthesisers. §3.1 sets up the formalism that frames the rest of the chapter. §3.2 and §3.3 define the process score and its weights; §3.4 defines the cleanliness sub-score the process score consumes. §3.5 defines the divergence-point detector, and §3.6–§3.9 define the four synthesisers that produce alternative reference trajectories. The validator that admits these alternatives as legitimate same-end-state reorderings is described in §3.10. §3.11 describes the rater study used to validate the manifest labels in Chapter 5.

This chapter focuses on what the algorithms do and why they are correct. How they are made fast in practice - the JDT batch-session reuse, single-worktree borrow, and git-checkout backtracking - is covered in Chapter 4 (§4.6).

3.1 Formal definitions and notation

The formal definitions in this section build on the state/action/trace framing from §2.1. A refactoring trajectory is a sequence

$$\tau = (s_0, a_0, s_1, a_1, \dots, a_{T-1}, s_T),$$

where each s_t is a code snapshot (a git commit SHA paired with the working-tree contents) and each a_t is a developer action that turns s_t into s_{t+1} . Actions are either IDE-supported refactoring operations or manual edits; in practice a single action can carry both a structured refactoring spec (recovered by RefactoringMiner [7]) and some leftover unstructured edits. We treat this as one action with two parts rather than two separate actions.

The object actually being evaluated is the analysis-report structure produced by the tool. An *analysis report* for a trajectory τ is a tuple $(\tau, \mathcal{M}, \mathcal{D}, \mathcal{A})$, where:

- $\mathcal{M} = \{m_0, m_1, \dots, m_T\}$ is a per-state metric assignment. Each m_t contains the cleanliness sub-score $C(s_t)$ together with the raw signals it is built from (PMD violations, CPD duplications, Chidamber–Kemerer measures, build/test outcomes, the action’s refactoring spec, and commit and test event metadata).
- $\mathcal{D}$ is the set of divergence points the detector emits for τ (§3.5).
- $\mathcal{A}$ is the set of alternative reference trajectories synthesised at those divergence points (§3.6 onwards). Each $\tau^* \in \mathcal{A}$ has the same tuple shape as the user’s report, so alts can be scored under the same machinery.

The process-quality score $J(\tau)$ is computed over the full trajectory using $\mathcal{M}$, and is the focus of §3.2–§3.3. Table 3.1 summarises the notation used throughout the chapter.

3.2 The process score

The process score $J(\tau)$ is the main thing this chapter defines. It combines six weighted terms across the trajectory and clamps the result to a $[0, 100]$ range so trajectories of different lengths

Symbol	Meaning
$\tau = (s_0, a_0, s_1, \dots, a_{T-1}, s_T)$	Refactoring trajectory
s_t	State (code snapshot) at step t
a_t	Action at step t (IDE refactor or manual edit)
$J(\tau)$	Process-quality score, clamped to $[0, 100]$
$C(s)$	Cleanliness sub-score of state s
$\Delta C(\tau) = C(s_T) - C(s_0)$	Cleanliness gain across trajectory
$\beta = 50$	Baseline anchor in the process score
W_x	Process-score weight for term x
r_b, r_{st}, r_{mi}	Laplace-smoothed rate terms (broken, skip-tests, manual-when-IDE)
$n_{cg}(\tau)$	Commit-gap event count
$\sigma_l(\tau)$	Step-savings bonus for alternative trajectories
DP	Divergence point
τ^*	Counterfactual reference trajectory
$\mathcal{K}$	Divergence-kind set = {ORDERING, MANUAL_REFACTOR, REWORK, HYGIENE}

Table 3.1: Notation used throughout the methodology chapter.

can be compared on the same scale:

$$J(\tau) = \text{clip}_{[0,100]} \left(\beta + W_G \Delta C(\tau) - W_B r_b(\tau) - W_{ST} r_{st}(\tau) - W_{MI} r_{mi}(\tau) - W_{CG} n_{cg}(\tau) + W_L \sigma_l(\tau) \right) \quad (3.1)$$

where:

- $\beta = 50$ is the baseline anchor. A trajectory with neither cleanliness gain nor process damage scores 50.
- $\Delta C(\tau) = C(s_T) - C(s_0)$ is the cleanliness gain from start to end (§3.4).
- $r_b(\tau) = b_{ms}(\tau)/e_{ms}(\tau)$ is the time-weighted broken fraction: the proportion of trajectory wall-clock time during which the build or test suite was broken.
- $r_{st}(\tau)$ and $r_{mi}(\tau)$ are Laplace-smoothed event-rate terms of the form $r_\bullet(\tau) = (k+1)/(n+2)$, where k is the number of penalised events and n the number of opportunities. Both default to zero when there have been no events of either polarity yet, so a single bad-out-of-one event does not trigger a full penalty.
- $n_{cg}(\tau)$ is the commit-gap event count: one event per stretch of ≥ 6 green-refactor checkpoints between user commits.
- $\sigma_l(\tau)$ is the alt-only step-savings bonus. It only contributes when τ is a synthesised counterfactual that reaches the same end state in fewer steps than the user.

Three design choices shape this formula. First, an *anti-gaming axiom*: the broken-state penalty W_B has to outweigh any single-step cleanliness gain $W_G \cdot \Delta C$ a developer could plausibly engineer by trading transient correctness for structural improvement. The severity ordering $W_B : W_{ST} : W_{CG} = 28 : 14 : 7$ encodes this in the production weights of §3.3. Second, the Laplace smoothing keeps the rate terms from misbehaving on very short trajectories where one bad event would otherwise dominate. Third, the $[0, 100]$ clamp makes scores comparable across trajectories of different lengths, but it does introduce saturation: when a user’s trajectory is already near 100, any alt’s improvement gets compressed against the ceiling. The sensitivity experiment in Chapter 5 tracks this and reports it as a known limitation of the score formula.

3.3 Process-score weights

Table 3.2 lists the six production weights of Equation (3.1), the literature that backs each one, and a forward reference to the ablation results in Chapter 5. The rightmost column gives the magnitude recovery of each term when it is the only term active in the formula - a single number from the full power-set ablation that captures how much of the production score’s signal each term carries on its own.

Term	Weight	Cited justification	Δ recovery (solo)
W_G (gain)	50	Cedrim et al. 2017 [36]; Paixão et al. 2017 [18]	0.000
W_B (broken)	28	Paixão et al. 2017 [18]; Gautam et al. 2025 [9] (FM #2)	0.348
W_{ST} (skip-tests)	14	Murphy-Hill et al. 2009 [5]	0.093
W_{MI} (manual)	11	Negara et al. 2013 [37]; Vakilian et al. 2012 [38]	0.357
W_L (length)	11	Harman & Tratt 2007 [14]; Mohan & Greer 2019 [15]	0.342
W_{CG} (commit-gap)	7	Tao & Kim 2015 [39]; Di Biase et al. 2019 [40]; Herzig & Zeller 2013 [41]; Kudrjavets et al. 2022 [42]	0.126

Table 3.2: Production weights for Equation (3.1), the literature each weight rests on, and the magnitude recovery when only that term is active in the formula (full power-set ablation, $2^6 = 64$ variants per fixture; see Chapter 5).

The rest of this section walks through each weight. Two of them (W_{MI} and W_{CG}) have a citation gap: the literature backs the general idea of the penalty but does not directly measure the specific differential the weight is supposed to capture. I flag this openly below.

W_G (gain, 50). The main positive term: a unit of cleanliness gain at the end of the trajectory adds 50 to $J(\tau)$. Cedrim et al. [36] found that refactoring often makes structural metrics no better or even slightly worse, which means the score should reward improvement explicitly rather than assume it. Paixão et al. [18] model the trade-off between disruption and improvement as something developers actively manage, which sets the size of this positive term in relation to the penalty terms below.

W_B (broken, 28). The largest single penalty term. Paixão et al. [18] show that developers care a lot about avoiding disruptive intermediate states - they are willing to give up roughly a quarter of the available metric improvement to avoid them. The weight is *time-weighted* rather than binary: $r_b(\tau) = b_{ms}/e_{ms}$, so a brief broken state is penalised proportionally less than a long one. The numerator b_{ms} counts only wall-clock time during which the build or tests are actually failing - not total refactoring time, and not time the developer spends thinking between changes. A developer who pauses to think carefully before making a clean change is not penalised; a developer who leaves the build broken for five minutes is. Time-weighting also makes the broken term robust to the choice of granularity. The state/action/trace formalism in §3.1 leaves what counts as a “state” deliberately flexible - a state can be a commit, an IDE event, or a developer-defined checkpoint - and a binary “was this state broken” counter would change by a large factor across those choices (count once per commit vs. once per keystroke and the broken-state count looks very different). Wall-clock time spent broken does not depend on the sampling rate, so $r_b(\tau)$ stays meaningful at any granularity. The time-weighting choice also lines up with Gautam et al.’s 2025 RefactorBench analysis [9]: their failure-mode #2 shows that strictly rejecting any intermediate broken state hurts LM-agent performance, because some broken intermediates are genuinely necessary for multi-file edits. The anti-gaming rule means W_B has to outweigh the single-step gain $W_G \Delta C$ for any plausible ΔC .

W_{ST} (skip-tests, 14). Penalises composite refactoring windows (60-second batches) that finish without any test run in between. The 60-second cut-off comes from Murphy-Hill et al. [5]: their empirical work on how developers actually refactor anchors test cadence at the batch level rather than the per-step level. I set the weight at half W_B : skipping tests is weaker evidence of process damage than an observably broken state, but it is still a signal the score should track.

W_{MI} (manual-when-IDE, 11). Penalises steps where the developer performed a refactoring by hand even though the IDE could have done it automatically. Negara et al. [37] show that this is a common case: across 23 developers and 5,371 refactorings, developers preferred to do them manually even when automated equivalents existed. The reliability argument comes from

Mens & Tourwé [4]: refactoring tools enforce static precondition checks that manual edits do not. Vakilian et al. [38] add nuance by showing that even automated refactorings can be misused, which is why this penalty is graded rather than binary. I am not aware of any published study that directly measures the fault-rate gap between manual and IDE refactoring; the specific weight value is therefore not fitted to data but sits within the severity ordering set out above. Chapter 5’s sensitivity and ablation experiments check whether that ordering holds up.

W_L (length, 11). This one is a bonus, not a penalty: it adds to the score of alternative trajectories that reach the same end state in fewer steps than the user. Harman & Tratt [14] and Mohan & Greer [15] treat “fewer refactorings” as an objective on equal footing with quality in multi-objective search-based refactoring, and recent many-objective work in the same lineage continues to treat refactoring effort as a first-class Pareto objective [25]. I deliberately do not penalise the user for a longer trajectory: a longer user trajectory might just mean a bigger task, not a worse process, and conflating the two would punish the user for the size of the work. The bonus only applies when the synthesiser produces an alt that hits the same end state in fewer steps.

W_{CG} (commit-gap, 7). Penalises long stretches of green-refactor checkpoints (refactor + tests pass) without an intervening commit. The argument comes from the change-decomposition literature: Tao & Kim [39] show that splitting composite changes into separate commits lets reviewers do a better job in the same amount of time, and Di Biase et al. [40] confirm in a controlled experiment that decomposed changesets reduce false-positive review noise. Herzig & Zeller [41] show that tangled commits inflate downstream defect attribution errors. I am deliberately *not* claiming a throughput benefit: Kudrjavets et al. [42] find that PR size does not correlate with time-to-merge across 1.25M PRs in 10 languages. So the real gain from smaller commit gaps is review *comprehensibility* and *rollback locality*, not faster merges. I am not aware of any published study that directly measures commit cadence *during refactoring* as a predictor of rollback or review cost; the specific weight value is therefore axiomatic in the severity-ordering sense and validated empirically in Chapter 5.

3.4 The cleanliness sub-score

The cleanliness sub-score $C(s)$ in Equation (3.1) combines six per-state quality signals into a single normalised value in $[0, 100]$. I picked the signals on the principle that no single quality measure is reliable on its own across different contexts (§2.3); combining several complementary signals reduces sensitivity to the noise of any one tool. Table 3.3 lists each sub-signal, its cited basis, and how it is computed.

Sub-metric	Weight	Cited basis	Computation source
Cognitive complexity	1.0	Campbell 2018 [43]; Muñoz Barón et al. 2020 [44] (partial validation); Lavazza et al. 2023 [45] (contradicting)	Per-method mean
Coupling (CBO)	1.0	Chidamber & Kemerer 1994 [46]	CK library, per-class mean
Duplication	1.0	Heitlager et al. 2007 [47]; Fowler 1999 [2]	CPD-detected cloned lines on touched files
Readability	1.0	Buse & Weimer 2010 [48] (feature taxonomy only - <i>not</i> the trained model)	Five-feature blend, line-weighted
Smells	1.0	Marinescu 2004 [49]; Arcelli Fontana et al. 2016 [21]; Moha et al. 2010 [19]	PMD violation count on touched files
Cohesion (TCC)	1.0	Bieman & Kang 1995 [50]	Per-class mean, dropped if < 2 methods

Table 3.3: The six sub-signals composing the cleanliness sub-score $C(s)$. All weights are uniform 1.0 following Laplace’s principle of insufficient reason (see composition rule below); the production weight bundle in the implementation reflects this.

Composition rule. Each raw sub-signal is min-max normalised against the main trajectory’s range, inverted for “lower is better” measures (cognitive complexity, coupling, duplication, smells). Sub-signals with degenerate ranges or missing data on the trajectory are dropped and the remaining weights renormalised. Alternative-trajectory checkpoint values are clamped to the unit interval against the main trajectory’s range, so an extreme alt cannot shift the main trajectory’s normalisation. The composite is a weighted sum, then scaled to $[0, 100]$. Wagner et al.’s Quamoco framework [51] and the QMOOD quality model [52] both use this kind of layered weighted-sum aggregation, which is a defensible choice when there is no calibration data to fit the weights against.

Uniform sub-weights, by deliberate choice. I set all six sub-weights to 1.0 because there is no published calibration data that tells me how to weight them differently for refactoring trajectories. Equal weighting is the default in the absence of such data - Laplace’s principle of insufficient reason. The sensitivity experiment in Chapter 5 confirms that the divergence-point ranking is largely insensitive to these cleanliness sub-weights: only a small fraction of cleanliness perturbations reshuffle the ranking, an order of magnitude lower than for the process-score perturbations.

Caveats deliberately disclosed. Two of the sub-signals have weaker empirical backing than the others. I want to flag both honestly:

- *Readability.* I use Buse & Weimer’s feature taxonomy [48] (line length, indentation, identifier characteristics) as a feature blend, but *not* their trained readability model. Calibrating their trained model to the trajectory-evaluation setting would need domain-specific data I have not collected. The feature blend without the trained model is a weaker signal but it is still defensible as one of six components.
- *Cognitive complexity.* Cognitive complexity is empirically contested. Muñoz Barón et al. [44] re-analysed 24,400 human understandability ratings and partially validated Campbell’s claim, finding significant positive correlation with comprehension time and subjective ratings; correlation with correctness was weak. Lavazza et al. [45], on the other hand, find that cognitive complexity predicts understandability about as well as traditional size and cyclomatic-complexity measures, with negligible additional explanatory power. I include cognitive complexity as one of six normalised sub-signals rather than as a dominant predictor, consistent with the uniform composition and the contested empirical status of the measure.

3.5 The divergence-point detector

A *divergence point* is a step in the user’s trajectory where an alternative trajectory exists that scores strictly better than the user’s under the process score. The detector emits divergence points of four kinds:

$$\mathcal{K} = \{\text{ORDERING, MANUAL_REFACTOR, REWORK, HYGIENE}\}.$$

Each divergence point has four fields: a *kind*, an *anchor step* (the user step where divergence occurs), a *magnitude*, and per-kind context (the reorder window for ORDERING; the replaced refactoring identifier for MANUAL_REFACTOR; the originating-terminal step pair, file, scope, and reverted-line count for REWORK; the hygiene sub-kind and stretch length for HYGIENE).

The magnitude is uniform across all four kinds:

$$\text{magnitude} = J(\tau_{\text{final}}^*) - J(\tau_{\text{final}}),$$

where τ_{final}^* is the alt’s process score *rolled forward through the user’s post-convergence activity* - i.e. the user’s trajectory with the alt’s path substituted in at the divergence and the rest of the trajectory unchanged - and τ_{final} is the user’s actual trajectory-final score. The comparison is therefore at the user’s trajectory-final state on both sides, so the magnitude answers a single question for every kind: “*if you had taken this one different path at the divergence point and continued with the rest of your trajectory unchanged, by how many process-score points would your final score have moved?*”

For REWORK specifically, the magnitude is non-trivial even though the alt reaches the same end state as the user: the alt has *fewer* steps because the round-trip work is elided, so the W_L step-savings bonus contributes positively to the alt’s score. The reverted-line count is still attached to

the divergence point as a separate field (`reworkLineCount`) for explanatory use on the dashboard, but it is no longer the ranking magnitude.

Before emitting a divergence point, the detector runs kind-specific filters. Three threshold constants gate detection: `MIN_REWORK_LINES = 2` drops single-line REWORK noise; `MIN_COMMIT_GAP = 6` requires at least six green-refactor checkpoints between commits before `COMMIT_GAP` fires; and the 60s composite-window boundary [5] defines the refactoring batch boundary used by both ORDERING and HYGIENE.

3.5.1 ORDERING detection

ORDERING detection works on bracket-validated windows of consecutive refactoring steps (§3.10). For each window, the synthesiser enumerates valid permutations of the steps - only those whose terminal AST matches the user's, established by the audit in §3.6 - and computes the process-score delta of each permutation against the user's. If any permutation strictly beats the user, a divergence point is emitted at the window's terminal step with the best-scoring permutation as the surfaced alternative.

3.5.2 MANUAL_REFACTOR detection

A `MANUAL_REFACTOR` divergence point is emitted at any user step where (i) the action was performed by hand, and (ii) the resulting diff matches the structural pattern of an IDE-supported refactoring. The pattern match is established by RefactoringMiner-style mining of the diff [7]: if a non-null IDE-mappable spec is recovered from the manual edit, the step is a candidate. Adjacent candidate steps that share the same (`fromSha`, `toSha`) bracket are grouped into a single composite; the synthesiser (§3.7) builds the alt by applying the recovered spec via the IDE-equivalent pathway. The pipeline wraps the Eclipse JDT refactoring libraries to apply these specs, and where the user's IDE (e.g. IntelliJ) and JDT disagree on the exact post-state of a refactoring (whitespace, import ordering, default-method positioning), the residual gap is closed with a manual git patch so the alt's terminal state aligns with the user's. The engineering details of this wrap-and-patch layer are described in §4.6.

3.5.3 REWORK detection

REWORK detection looks for pairs of changes within a trajectory where content is added and later removed (or removed and later re-added) such that the two halves net to a no-op, modulo whitespace. The detector parses each step's unified diff into hunks, resolves each hunk to its enclosing method or class scope, content-hashes the affected lines (whitespace-normalised), and greedily pairs add/remove hunks that share the same (`file`, `scope`, `content-hash`). Each paired (`originating step`, `terminal step`) becomes a REWORK candidate; pairs below the `MIN_REWORK_LINES` floor are dropped as line-noise.

3.5.4 HYGIENE detection

HYGIENE detection has two sub-kinds. `TESTS_SKIPPED` fires once per composite window (60-second batch [5]) that contains at least one refactoring step and no successful test run in between. `COMMIT_GAP` fires once per stretch of ≥ 6 green-refactor checkpoints (checkpoints whose tests passed and whose refactoring applied cleanly) without an intervening user commit. Both sub-kinds emit one divergence point per finding.

3.6 Reorder synthesiser

The reorder synthesiser builds alternative trajectories by reshuffling the user's refactoring steps into a different order while still reaching the same end state. Correctness is established in six steps: window-splitting on validator verdicts, dependency-DAG construction over the program entities each step reads and writes, single-static-assignment (SSA) versioning across renames, topological enumeration, depth-first search over a prefix trie of candidate permutations, and a terminal AST-equivalence audit. The engineering choices that make the search tractable - single borrowed worktree, single JDT batch session, git-checkout backtracking - are described in §4.6.

Window splitting on validator verdicts. Only bracket-validated windows (§3.10) are eligible for reordering. Brackets with verdict `AST_DIVERGED` (the user’s terminal AST differs from the synthesiser’s terminal AST under straight replay) or `REFACTOR_FAILED` (some refactoring step could not be applied) are split into singleton windows, because reordering inside them risks producing an unsafe alternative. `AST_DIVERGED` commonly arises from systematic differences between the user’s IDE (IntelliJ) and the Eclipse JDT libraries the pipeline wraps to apply refactorings: the two engines disagree on details such as whitespace, import-statement ordering, or where a default method is placed within a class, producing terminal ASTs that are equivalent in intent but not byte-identical. The wrap-and-patch layer described in §4.6 closes these gaps for accepted brackets; brackets where the residual gap cannot be patched safely get the `AST_DIVERGED` verdict and are not reordered.

Dependency-DAG construction. Each refactoring step carries a structured *spec* describing the entities it reads, writes, creates, or deletes. The synthesiser builds a directed acyclic graph $G = (V, E)$ where vertices are spec instances and $(u, v) \in E$ if v depends on u ’s effects - specifically, if v ’s read-set intersects u ’s write-set, or v ’s target was created by u . The DAG captures every dependency constraint a valid permutation has to respect.

SSA versioning across renames. If a method is renamed at step t from `foo` to `bar`, it shows up in later steps under its new name, which breaks simple name-based equality across the trajectory. The synthesiser threads version identifiers through the DAG using single-static-assignment-style versioning: each rename produces a fresh version, and subsequent steps that target the renamed entity bind to the right version. This handles trajectories like Extract Method followed by Rename Method on the extracted method.

Topological enumeration. The synthesiser enumerates topological orderings of the DAG via straightforward recursive backtracking. Each enumeration is a candidate permutation of the window’s steps. In practice the DAG sizes are small (most brackets contain 2–8 steps), so the enumeration terminates quickly.

Depth-first search over a prefix trie. Permutations share prefixes - two different orderings that agree on their first k steps share the first k levels of a trie. The synthesiser applies steps in order along trie edges, reusing the JDT project state across the shared prefix and only re-applying the divergent suffix when descending into a new sub-tree. This saves the work of re-applying shared prefixes, which would otherwise dominate the cost of naive enumeration.

Terminal AST-equivalence audit. For each permutation that reaches a terminal state, the synthesiser hashes the affected `.java` files with a Java AST hasher (which canonicalises method-declaration order and other syntactic differences that do not change behaviour) and compares them to hashes of the user’s terminal state computed via `git show`. Only permutations whose terminal AST hashes *exactly match* the user’s count as legitimate reorderings. This is the correctness guarantee: by construction, an admitted reorder reaches the same end state as the user.

The admitted permutations are then scored under Equation (3.1). The highest-scoring permutation that strictly beats the user’s becomes the alternative trajectory for the ORDERING divergence point at the window’s terminal step.

3.7 Manual-refactor synthesiser

The manual-refactor synthesiser builds an alternative trajectory in which the user’s manual refactoring is instead applied via the IDE’s automated pathway. The synthesiser borrows a worktree at the originating commit `fromSha`, applies each detected spec via its IDE-equivalent operator inside a single JDT batch session, and then runs a residual three-way merge to layer the user’s non-refactoring edits (for example, unrelated bug fixes interleaved with the refactor) on top of the synthesised refactor. The resulting tree is committed to a shadow repository.

The residual merge is important: it makes sure the alt trajectory keeps all of the user’s non-refactoring edits. That way the comparison is about *how* the user reached the end state (by hand vs. via the IDE), not about *whether* they reached it. Without the residual merge, the alt would

drift away from the user’s end state on the non-refactoring axis and the process-score comparison would be confounded.

3.8 Rework synthesiser

The rework synthesiser builds an alternative trajectory in which the originating-terminal hunk pair identified by REWORK detection (§3.5.3) is skipped. For each pair, the synthesiser borrows a worktree at the originating step’s pre-state and generates a sequence of surgical patches covering the [originating, terminal] window. Each patch is applied via `git apply` and committed; branch references are force-updated to point at the new heads. The operation is atomic per trajectory: either all patches in the window apply successfully, or the alt is thrown away.

The surgical replay preserves all non-reworked content. Because the alt skips the round-trip work, it reaches the same end state as the user in fewer steps; the W_L step-savings bonus therefore contributes positively to the alt’s trajectory-final score, which drives the divergence-point magnitude (§3.5). The reverted-line count is preserved on the divergence point as a separate field (`reworkLineCount`) for explanatory use on the dashboard.

3.9 Hygiene synthesisers

The hygiene synthesiser builds alternative trajectories that represent the user observing better process hygiene at points where they did not, without changing the code itself. There are two sub-kinds. For TESTS_SKIPPED, the alt represents a trajectory in which the user ran the test suite at the anchor checkpoint and it passed. For COMMIT_GAP, the alt represents a trajectory in which the user committed at the anchor checkpoint, breaking what would otherwise have been a long stretch of uncommitted refactor checkpoints.

Both sub-kinds are synthesised by adjusting the anchor checkpoint’s process metadata rather than by changing the code, because hygiene is fundamentally a *process* property: when did the developer run tests, when did they commit. The alt’s code state stays identical to the user’s at every checkpoint; only the metric stream changes. Re-scoring the trajectory with the adjusted metadata and comparing trajectory-final scores under the magnitude semantic of §3.5 gives the cost of the missing hygiene event in process-score points.

The metadata-adjustment approach is the honest representation of what a hygiene-improved counterfactual actually looks like: a developer running tests at the anchor checkpoint, or committing earlier, does not change the code at that checkpoint, only the metric stream that accumulates downstream from it.

3.10 Validator and behaviour preservation

Before reordering anything, the reorder synthesiser needs a behaviour-preservation guarantee on the user’s window: replaying the user’s specs in their original order has to reach the user’s terminal state. If it does not, reordering is unsafe. The bracket-level validator establishes this guarantee. For each (`fromSha`, `toSha`) bracket, the validator borrows a worktree at `fromSha`, replays the user’s specs in their original order inside a single JDT batch session, and runs two checks:

- The set of `.java` files changed by the replay matches the set changed in the user’s `git diff fromSha..toSha`.
- The terminal AST hash of every changed file matches the hash of the user’s terminal state.

The verdict is VALID if both checks pass, AST_DIVERGED if hashes mismatch, and REFACTOR_FAILED if any spec fails to apply. Only VALID brackets become reorder windows; the other two verdicts get split into singleton windows (§3.6). The engineering choices that let the validator bracket-replay efficiently at scale - single batch session, single worktree borrow, AST-hash computation via `git show` rather than a second worktree - are described in §4.6.

A practical limitation worth flagging: the pipeline applies refactorings through Eclipse JDT, but the user’s actions are recorded from IntelliJ. The two refactoring engines sometimes produce *structurally* different terminal ASTs for the same nominal operation, particularly around how the

method body and its call sites are wired together. Two examples currently handled by the patch layer:

- *Static modifier on Extract Method.* JDT only adds `static` when it is required by the extracted body; IntelliJ adds it whenever the body permits. The patch layer adds `static` post-extract when the user’s version had it, so the two terminal ASTs match.
- *Return-vs-void shape on Extract Method.* When the extracted selection assigns to an outer-scope variable that is read after the selection, IntelliJ generates a method that returns the new value and rewrites the call site to assign it (`x = f(x)`), whereas JDT under-detects this liveness in conditional branches and instead generates a `void` method that takes the variable as a parameter and assigns to it locally (`f(x)` at the call site). The patch layer detects this pattern and rewrites JDT’s output to match IntelliJ’s return-and-assign shape.

Where the patch layer covers the divergence pattern, the bracket is admitted as `VALID`. Where it does not, the bracket falls to `AST_DIVERGED` even though the underlying refactoring was correct, and the synthesiser conservatively splits the window into singletons rather than risking an unsafe reorder. The current patch layer handles the most common divergence patterns observed during corpus recording; extending it to cover the long tail of structural JDT-versus-IntelliJ differences is left as future work and is the primary source of the `ORDERING` recall gap reported in Chapter 5.

3.11 Rater design

The labelled corpus used to evaluate the divergence-point detector in Chapter 5 was annotated against a multi-label schema covering the four divergence kinds in $\mathcal{K}$. To check the schema against someone other than the author, a rater study was conducted in which two independent annotators labelled the corpus against the same schema without seeing the author’s labels. The per-kind Cohen’s κ agreement statistics and the protocol used to brief the raters are reported in this section once the rater study has been conducted, and are used in Chapter 5 to bound the empirical claims made against the corpus.

Chapter 4

Tool Architecture

This chapter describes the engineering side of the tool. The methodology chapter argued for why each algorithm is correct; this chapter argues for why each one is fast. It walks through the pipeline in order: event capture in the plugin (§4.2), shadow-repo reconstruction and worktree allocation (§4.3), per-checkpoint metrics calculation (§4.4), the Eclipse JDT refactoring engine that every synthesiser leans on (§4.5), and the four counterfactual-trajectory synthesis strategies (§4.6). The split of the pipeline into an expensive offline phase and a cheap replay phase is described in §4.7, and the dashboard that consumes the resulting report in §4.8.

4.1 Architecture at a glance

The tool consists of four JVM modules (`:ide-plugin`, `:analysis`, `:refactoring-bundle`, `:shared`) plus a standalone React/TypeScript dashboard (`dashboard/`):

- `ide-plugin/` – an IntelliJ plugin that records the developer’s session as a stream of typed events and uploads it to the analysis server.
- `analysis/` – the Kotlin pipeline; it ingests sessions, reconstructs a shadow git repo, mines and synthesises trajectories, computes per-checkpoint metrics, and emits an `AnalysisReport`.
- `refactoring-bundle/` – a headless Eclipse JDT distribution wrapped in an embedded Equinox OSGi framework; the analysis module talks to it across the OSGi classloader boundary (details in §4.6).
- `dashboard/` – a React/TypeScript single-page app that renders the `AnalysisReport` (details in §4.8).
- `shared/` – cross-wire types (`Session`, `TraceEvent`, `SessionMetadata`, `FileSnapshot`, `EventType`) used on both the plugin and the analysis side.

Figure 4.1 shows the dataflow at a glance.

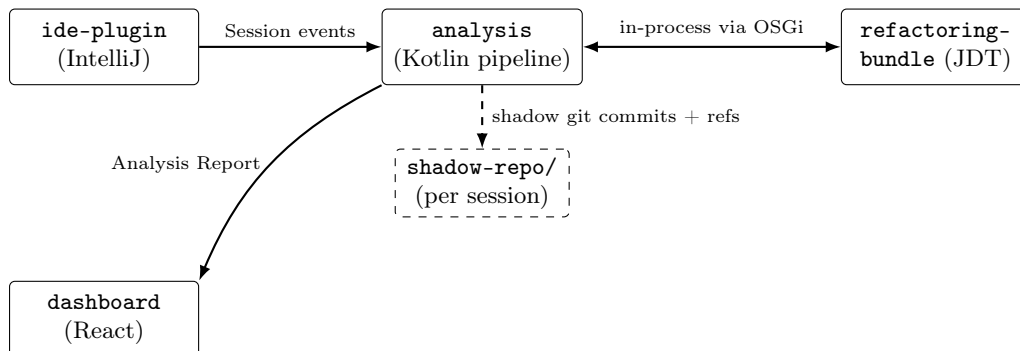


Figure 4.1: High-level dataflow between the recording side (IDE plugin), the analysis pipeline, the embedded Eclipse JDT engine, and the dashboard. The shadow repo holds every reconstructed state – both user checkpoints and synthesised alternatives – as git commits in a per-session repository.

4.2 Event capture

The IntelliJ plugin is responsible for capture the IntelliJ events which are emitted during the user’s refactoring process. The captured event types cover editor activity (files opened, closed, saved, modified, renamed, moved), refactoring activity (refactoring start / finish), build activity (build start / finish), test activity (test-run start / finish with pass and fail counts), git activity (one event per new commit observed in the underlying repository), and the session lifecycle.

Editor activity is the noisiest source, so keystroke-level events are not emitted directly. A debouncing layer accumulates keystrokes into an *edit burst*: when a quiet window elapses without further typing, a single burst event is emitted carrying the file’s post-edit contents and the structured set of program elements touched.

Although a handful of the fine-grained file-level events (file open, save, and similar) are not consumed by any downstream stage, they are recorded because their capture is essentially free at the plugin layer, and they remain obvious candidates for future work on navigation patterns, file dwell time, and read-versus-edit behaviour.

4.3 Shadow repo reconstruction and worktree pool

Analysing a user’s trajectory requires the ability to reconstruct, and cheaply check out, the exact state of the codebase at any point during the session. This is made possible by the Shadow Repo and Worktree Pool

Shadow repo reconstruction. The shadow repo is built by replaying the normalised event stream into a fresh per-session git repository, one commit per state-bearing event. The baseline commit is the initial source snapshot taken at session start; each subsequent event that carries file changes applies its snapshots to the working tree, stages them, and commits. No-op events – those that carry no file changes, or whose staged diff is blank-line-only – map to the previous SHA without producing an empty commit, so the unique-SHA count is typically much smaller than the event count. The output is a per-event map from event id to SHA, which gives every downstream stage a stable (from-SHA, to-SHA) pair to check out.

Worktree allocation. Borrowers – the validator, the reorder synthesiser, the IDE-replay runner – ask for a fresh worktree at a target SHA, run their operation, and release it back when done. Fresh-per-borrow rather than reuse-across-SHAs is deliberate: it guarantees no leftover build artefacts, stale Gradle cache, or other cross-SHA pollution between consecutive borrows.

4.4 Metrics calculation

For every unique state in the shadow repo the pipeline computes a checkpoint metric record: the build and test outcome, the cleanliness signals (PMD violations, CPD duplications, code readability, and Chidamber–Kemerer structural measures), and the file-level diff against the previous checkpoint.

The build and test outcomes are produced by checking the project state out into a worktree and invoking the project’s own Gradle build and test tasks against it, recording for each checkpoint whether the build succeeded and whether the test suite passed. This is the stage that dominates the wall-clock cost of the pipeline; everything else is essentially free. Two engineering choices on the build side bring this cost down. First, all checkpoints share a single persistent worktree (the only stage that bypasses the fresh-per-borrow rule of §4.3), and the pipeline walks the unique SHAs in order using detached checkouts to switch between them. The build directory of that worktree is preserved across SHAs, so the Gradle daemon, the Gradle build cache, and incremental compilation all have prior state to reuse on the next checkpoint. Second, an earlier design fanned this work out across multiple fresh worktrees in parallel; on the corpus sessions this lost to the single-worktree-with-reuse approach by roughly 2×, because daemon warmup and cache reuse dominate fan-out at this scale.

The static-analysis signals are tracked across checkpoints rather than just computed per checkpoint. Each PMD violation and CPD clone group is threaded through the unified diffs so the

report can identify when a violation was first introduced, when it was resolved, and which checkpoint transition is responsible. This is what lets the dashboard show, for any refactoring step, the smells it introduced or eliminated.

4.5 Applying refactorings via Eclipse JDT

Synthesising counterfactual trajectories requires actually applying refactorings to the code, not just describing them. Eclipse JDT is the only Java refactoring engine with both the breadth (extract, inline, move, rename, pull-up, push-down, and so on) and the maturity to serve as ground truth for an analysis pipeline. JDT was designed to run inside an Eclipse workbench, however; the supported route to use it elsewhere is to host an embedded Equinox OSGi framework in the same JVM and load the JDT bundles into it. The Eclipse JDT FAQ [53] distinguishes two headless paths: bare-classpath usage for parser and AST features, and a runtime-workbench (i.e. Equinox-hosted) deployment for anything workspace-dependent. Refactorings, which depend on a workspace-resident project model, fall in the second category. The reference implementation of this pattern maintained by the Eclipse Foundation is the Eclipse JDT Language Server [54], characterised by its maintainers as “a small, headless Eclipse JDT distribution”. To the author’s knowledge, no peer-reviewed paper documents the embedded-Equinox-in-a-non-Eclipse-JVM pattern; it is treated as engineering plumbing.

Batch sessions. JDT keeps an indexed project model in memory. Building that model from scratch – parsing every source file, building the classpath and type-binding caches – takes on the order of a second for a small fixture, and tearing it down and rebuilding between refactorings would dominate the wall-clock cost of any window that applies more than a handful of operations. The pipeline therefore wraps a whole window of refactorings in a single *batch session*: the project model is initialised once at the start of the window, held warm across every call inside it, and torn down at the end. Each individual refactoring inside the batch sees a fully indexed model and pays only the per-call cost.

Anchor resolution. A refactoring needs to know what to operate on – “the method at line 42, column 8” – but raw source coordinates are brittle, since any prior edit in the same file shifts subsequent line numbers. The pipeline addresses each target by a triple of line, column, and a canonical hash of the surrounding AST subtree. The hash makes the selection robust to formatting changes and to whole-file line shifts: even if line 42 has become line 47, the subtree at the original location can still be located by hash. The same canonical hash is computed on both the host side, when constructing the refactoring spec, and on the JDT side, when resolving the target, so the two agree on which AST node is meant.

4.6 Synthesising counterfactual trajectories

For each divergence point the detector emits, the pipeline synthesises a counterfactual reference trajectory: an alternative path between the same start and end states that scores higher under the process metric.

Reorder synthesis. A reorder window of k refactoring steps has up to $k!$ candidate orderings, and each ordering has to be applied against the same starting project state to check whether it reaches the user’s terminal AST (the methodology-level account is in §3.6). The synthesiser does not enumerate $k!$ orderings naively. It first builds a directed acyclic graph of must-come-before edges between the steps, derived from the reads, writes, produces, and consumes effects each refactoring spec declares, so the search is restricted to orderings the dependency lattice admits. It then folds every admissible ordering into a prefix trie and walks the trie depth-first: each forward edge applies one refactoring; each back edge rolls the worktree to the parent commit. Two orderings that agree on their first j steps share the first j trie nodes, so each unique prefix is materialised exactly once no matter how many leaves reuse it.

At each leaf of the trie – the terminal state of one full ordering – the synthesiser hashes the touched files’ ASTs into a canonical form and compares those hashes against the user’s terminal AST hashes, which are precomputed from the user’s own trajectory at the start of the window.

The hashing collapses cosmetic differences (whitespace, comment positioning, formatting variation) so two ASTs that are structurally identical produce the same hash. An ordering whose terminal hashes match the user’s is admitted as an end-state-equivalent alternative; one whose hashes do not match is discarded, since it produced a different end state even though every step in it was a valid refactoring application. This is what distinguishes the admitted alternatives from merely-plausible-looking ones.

Three engineering choices make this search tractable at corpus scale. First, the whole window runs against a single borrowed worktree, so forward edges, back edges, and the final AST audit all reuse the same on-disk checkout rather than paying the cost of a fresh worktree per ordering. Second, the whole window also runs inside a single JDT batch session (§4.5), so every refactoring inside it sees a warm project model and pays only the per-call cost. Third, back edges in the search roll the worktree back via a detached git checkout and then ask the JDT engine to re-stat its files via the same “files changed under us” pathway an IDE user exercises constantly.

Manual-refactor synthesis and the wrap-and-patch layer. A manual-refactor divergence point names a step where the user performed by hand what the IDE could have done in one operation (§3.7).

Identifying these steps requires first identifying the manual refactorings themselves, which is non-trivial: IDE-driven refactorings arrive pre-tagged in the captured event stream, but a manual refactoring is just a series of edit bursts whose net effect happens to constitute a structured refactoring. The pipeline recovers them by running RefactoringMiner [7] – a standard tool that detects refactorings between two code states by AST-level structural matching – over a sliding window of commit pairs in the shadow repo. The window matters because a manual refactoring may unfold across several intermediate edit bursts, each of which on its own looks like an unrelated change; only by widening the window to cover the full span does the refactoring become visible to the miner. Each detected refactoring is then cross-checked against the captured event stream: a detection whose window contains a matching IDE-emitted refactoring event is recorded as IDE-driven, and one whose window contains no such event is flagged as manual.

Once a manual refactoring is identified, the synthesiser builds the counterfactual by applying that refactoring through the JDT engine. The engineering complication is that the user records sessions in IntelliJ but the pipeline applies refactorings via JDT, and the two engines sometimes produce different terminal ASTs even when they agree on a refactoring’s semantics. A wrap-and-patch layer closes the residual gap. Two concrete divergences observed during corpus recording: JDT only adds the `static` modifier to an extracted method when the body requires it, while IntelliJ adds it whenever the body permits; and JDT under-detects outer-scope-variable liveness in conditional branches, producing a void-returning method where IntelliJ would produce a value-returning one. A host-side post-pass detects each pattern and rewrites the bundle’s output to match. Where the residual gap cannot be patched safely, the bracket is marked as diverged and not reordered: the wrap-and-patch layer expands the set of admissible brackets but does not fully bridge the two engines. A complementary empirical study [55] of 518 refactoring-engine bugs across Eclipse JDT, IntelliJ, and NetBeans found Extract-family refactorings to be the dominant bug class (28.76% of bugs studied), and the closest documented inter-engine divergence [56] is the Make-Static `final`-parameter disagreement between the same two engines.

Rework synthesis. A rework divergence point names two steps where the user added a chunk of code at one step and removed it at a later step (or vice versa) within the same logical scope (§3.8). Detection is content-addressed: each diff hunk is hashed against a normalised view of its contents (trailing whitespace trimmed, blank-line-only lines dropped), and chunks with matching hash and matching enclosing scope are paired greedily across the session. Two steps separated by many intervening edits will pair if their hashes match; there is no temporal-locality assumption between the originating and terminal steps.

Synthesis then applies a surgical patch at the originating step that skips the round-trip work, and replays the user’s intermediate edits on top. The complication is that those intermediate edits have shifted line numbers around the chunk, so the terminal-step surgery has to translate its line coordinates from the user’s tree (where the chunk reappears) to the synthesised tree (where the chunk was never written, so the file is several lines shorter). A per-file drift tracker maintains the divergence between the two trees as a set of zones; each intermediate user hunk shifts every zone

past it by the hunk’s net line delta, so each zone stays anchored at the same logical content as the user’s tree grows or shrinks around it.

After surgery, some of the intermediate checkpoints have no remaining content of their own – their only change was the wasted work the surgery has now excised – and others are left holding only cosmetic residue, such as blank-line edits that surrounded the reworked chunk. Empty checkpoints are dropped from the alt trajectory entirely rather than being committed as no-op steps; checkpoints whose surviving change is purely whitespace are folded into the preceding checkpoint. Both moves shorten the alt’s step count, which lifts its process score directly through the step-savings term defined in §3.2.

Hygiene synthesis. A hygiene divergence point names a process issue rather than a code issue: tests were skipped on a checkpoint that needed them, or the user stretched a long run of green-refactor checkpoints without committing (§3.9). The synthesised counterfactual does not change the code at all; it flips the relevant process flag on the anchor checkpoint and re-scores the trajectory. The alternative trajectory’s code is identical to the user’s at every step; only the metric stream differs. The resulting magnitude is the cost of the missing hygiene event in process-score points, measured directly against the same scoring formula that produced the user’s score.

4.7 Pipeline Phases

The pipeline is split into two phases. The first covers every stage above – event ingestion, shadow-repo reconstruction, metrics calculation, and synthesis – and takes a few minutes per session, dominated by the build and test runs of §4.4. Its output is a serialisable record of every raw signal the next phase needs. The second takes that record together with a choice of weights for the process-score formula and produces the final analysis report; it does no I/O, runs no expensive analyses, and finishes in milliseconds. The architectural value of the split is that the score formula’s weights enter only in the second phase, so perturbing them cannot invalidate any of the cached output from the first. The sensitivity experiment of Chapter ?? exploits this directly: it runs $12 \times 9 \times 45 = 4,860$ scoring configurations against the same cached intermediate output in roughly 2.6 seconds of wall-clock time, because each configuration only re-runs the second phase.

4.8 Dashboard

The dashboard is a single-page web app that consumes the analysis report and presents the session interactively. The trajectory chart plots the user’s process score over time, with the synthesised alternative trajectories overlaid as comparison lines so the developer can see how each counterfactual would have scored against their own. Clicking on a refactoring step opens a detail panel showing what the step did, which smells it introduced or eliminated, and which code-quality metrics shifted across it. Divergence points appear as flagged moments on the trajectory, each one a click-through to the counterfactual that scored higher. A separate panel summarises the highest-impact suggestions for the session as a whole, so the developer has both a per-step view and a session-level takeaway.

Chapter 5

Project Plan and Timetable (Jan–Jun 2026)

Work completed so far (as of 21 Jan 2026).

- Extensive related-work research completed (refactoring definitions, endpoint evaluation, smells/metrics, mining, sequence generation, IDE logging), with an initial BibTeX file in progress.
- Early drafts written for the Introduction and Background/Related Work chapters

Timeline in 5 phases (Jan → end of June).

- **Late Jan–Feb:** define refactoring *episodes* and system outputs (trace format, step definition, required explanations), pick an initial metric (*Metric v0*), and finalise the IDE plugin plan.
- **Feb–Mar:** implement the IDE plugin and begin collecting a first batch of refactoring episodes/traces using the plugin.
- **Mar–Apr:** build the offline analysis pipeline (episode → feature extraction → scoring) and implement baseline explanation outputs (per-step breakdown and “what got worse/why” style diagnostics).
- **Apr–May:** implement reference-trajectory generation (v1) and divergence detection (developer trajectory vs. reference trajectory), producing explanations of where/why the paths differ.
- **May–Jun:** run evaluation (human traces + simulated “unclean” variants, with optional open-source mining if feasible), consolidate results, and complete the final report write-up and polishing.

Chapter 6

Evaluation Plan

The main thing I need to show is that the system can take a real refactoring trace and produce explanations that a developer would actually find useful. Concretely, given (i) a repository, (ii) a chosen start and end revision, and (iii) an observed trace of intermediate snapshots, the system should (a) score the observed trajectory, (b) construct a plausible higher-scoring reference trajectory between the same endpoints, and (c) highlight a small set of divergence points with short “what changed / why it matters / what could have been done instead” explanations.

My primary plan is to collect refactoring episodes from humans using my custom IDE plugin, so I have traces in exactly the format the system is designed for. These episodes should include both IDE-supported refactorings and manual edits, and capture whatever signals I can log reliably (e.g., checkpoints/snapshots, refactoring commands, and build/test outcomes where available). This gives me realistic traces with the right granularity, without relying on commit history as the only source of “steps”.

On top of these real episodes, I will create controlled variants by deliberately injecting known bad-process patterns (e.g., unnecessary detours, rework, churn spikes, temporary quality regressions, or broken intermediate builds/tests). The point is to create a rough ground truth for where the divergence *should* be. I can then check whether the system finds these injected divergence points and whether the constructed reference trajectory avoids the injected detours while still reaching the same end state. I will measure this mainly by whether the reference reduces the “bad” signals I injected (e.g., effort/churn, temporary smell/metric spikes, intermediate build/test failures), rather than only by endpoint quality.

I am also considering mining additional episodes from open-source Java repositories, especially small and relatively self-contained refactoring PRs or multi-commit refactoring sequences, but I am not committing to that yet. If I do it, the goal would be to stress-test robustness to real-world messiness (mixed edits, inconsistent commit granularity, incomplete separation between refactoring and feature work) and to increase the diversity of codebases beyond what I can collect through the plugin alone.

I will also run a small human evaluation. The plan is to show participants a refactoring episode plus the system’s divergence explanations, and compare this against a few simple baselines: (i) endpoint-only before/after metrics, (ii) process plots that just flag the largest per-step regressions (no reference trajectory), and (iii) a weak “greedy/local” baseline that makes only local one-step improvements. Participants will rate which output better helps them understand what went wrong in the process and what they would do differently, and give a usefulness/actionability score. I will also collect short free-text comments so I can see what kinds of explanations felt convincing vs. annoying.

Chapter 7

Ethical Issues and Data Handling Plan

The main ethical considerations in this project come from collecting developer traces via an IDE plugin, and from the possibility that the tool could be misused as a form of monitoring. The technical work itself is not safety-critical, but the data involved can still be sensitive because it relates to how people work and may include source code artefacts.

If I collect refactoring episodes from human participants, I will use informed consent and make it clear what is logged (e.g., refactoring commands, checkpoints/snapshots, basic metadata like timestamps, and build/test outcomes) and what is *not* logged (e.g., personal accounts, unrelated browsing, or continuous keystroke-level recordings unless genuinely required). I will follow a data minimisation approach: only collect the smallest amount of information needed to reconstruct refactoring steps and compute the evaluation metrics. Participants will be able to withdraw, and I will delete their data if requested.

Source code can be commercially sensitive, even when the research goal is about refactoring rather than the code itself. To avoid IP issues, I plan to run participant studies on toy projects and/or open-source repositories, rather than proprietary code. If open-source repositories are used, I will respect licensing requirements and avoid re-publishing large code excerpts in the dissertation. If I later want to evaluate on any non-public codebase, I would only do so with explicit written permission and with clear constraints on what is stored and shared.

Even without names, traces can sometimes be identifying (e.g., file paths, commit messages, timestamps, distinctive code structure). For any stored traces, I will remove or hash direct identifiers (names, usernames, machine paths, repository URLs) and avoid publishing raw logs. In the dissertation, I will present results in aggregated form where possible (e.g., across episodes), and I will be careful not to “name and shame” individual developers or projects when discussing low-quality trajectories.

A tool that evaluates a developer’s “process quality” could be repurposed for workplace surveillance or performance management. Although that is not the intended use, it is important to acknowledge the risk. I will frame the tool as a developer support and coaching system (actionable feedback about code/process), not as a ranking system for people. In particular, I will avoid outputs that resemble employee scorecards or leaderboards, and I will focus on episode-level technical explanations rather than person-level judgements.

All collected data (traces and any derived metrics) will be stored securely with access limited to the project work. I will keep raw logs only for as long as needed to run experiments and compute results, and then retain only the derived, anonymised summaries required for analysis and write-up. If any data needs to be shared with a supervisor or included in an appendix, it will be anonymised and minimised to reduce the chance of leaking sensitive information.

Appendix A

First Appendix

Bibliography

- [1] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA, 1992.
- [2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [3] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.
- [4] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.
- [5] Emerson R. Murphy-Hill, Chris Parnin, and Andrew P. Black. How we refactor, and how we know it. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 287–297, 2009.
- [6] Miryung Kim, Danny Cai, and Sunghun Kim. An empirical investigation into the role of API-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 151–160, 2011.
- [7] Pouria Alikhanifard and Nikolaos Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. *ACM Transactions on Software Engineering and Methodology*, 34(2):Article 40, 2025.
- [8] Jean-Rémy Falleri and Matias Martinez. Fine-grained, accurate, and scalable source code differencing. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024.
- [9] Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [10] Islem Bouzenia and Michael Pradel. Understanding software engineering agents: A study of thought-action-result trajectories. 2025.
- [11] Eman Abdullah AlOmar et al. Software refactoring research with large language models: A systematic literature review. *Information and Software Technology / Journal of Systems and Software*, 2025. Article S0164121225004315.
- [12] Veselin Raychev, Max Schäfer, Manu Sridharan, and Martin Vechev. Refactoring with synthesis. In *Proceedings of the 2013 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 339–354. ACM, 2013.
- [13] Yu Lin, Xin Peng, Yumin Cai, Danny Dig, Li Zhang, and Wenyun Zhao. Interactive and guided architectural refactoring with search-based recommendation. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 535–546. ACM, 2016.
- [14] Mark Harman and Laurence Tratt. Pareto optimal search-based refactoring at the design level. *Journal of Systems and Software*, 80(4):522–534, 2007.
- [15] M. Mohan and Des Greer. Using a many-objective approach to investigate automated refactoring. *Information and Software Technology*, 2019.

- [16] Wang et al. 2018.
- [17] Nikolaos Nikolaidis et al. A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 2024.
- [18] Matheus Paixão et al. An empirical study of cohesion and coupling: Balancing optimisation and disruption. *IEEE Transactions on Evolutionary Computation*, 2017.
- [19] Naouel Moha et al. Decor: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering*, 2010.
- [20] Thiago Paiva et al. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development*, 2017.
- [21] Francesca Arcelli Fontana et al. Antipattern and code smell false positives: Preliminary conceptualization and classification. In *Proceedings of a workshop on code smells / antipatterns*, 2016.
- [22] Simone Scalabrino et al. A comprehensive model for code readability. *Journal of Systems and Software*, 2018.
- [23] Danilo Silva et al. Refdiff: Detecting refactorings in version histories. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2017.
- [24] Reudismam Rolim, Gustavo Soares, Loris D’Antoni, Oleksandr Polozov, Sumit Gulwani, Sherry Ge, and Neha Rungta. Learning syntactic program transformations from examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 404–415. IEEE, 2017.
- [25] Vittorio Cortellessa, Daniele Di Pompeo, Vincenzo Stoico, and Michele Tucci. Many-objective optimization of non-functional attributes based on refactoring of software models. *Information and Software Technology*, 2023.
- [26] Lerina Chen and Shinpei Hayashi. MORCoRA: Multi-objective refactoring recommendation considering review availability. *International Journal of Software Engineering and Knowledge Engineering*, 34(12):1919–1947, 2024.
- [27] Vahid Alizadeh, Marouane Kessentini, Ali Ouni, Mohamed Wiem Mkaouer, Mel Ó Cinnéide, and Yuanfang Cai. An interactive and dynamic search-based approach to software refactoring recommendations. *IEEE Transactions on Software Engineering*, 46(9):932–961, 2020.
- [28] Valentina Pantiuchina, Fiorella Zampetti, Simone Scalabrino, et al. Why developers refactor source code: A mining-based study. *ACM Transactions on Software Engineering and Methodology*, 29(4), 2020.
- [29] Germán Robredo, Isabel Gonzales, Yania Crespo, et al. What were you thinking? an llm-driven large-scale study of refactoring motivations in open-source projects, 2025. arXiv:2509.07763.
- [30] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yuan-dong Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.
- [31] Anna Maria Eilertsen. A study of refactorings during software change tasks. *Journal of Software: Evolution and Process*, 2024.
- [32] Kostadin Damevski, Thomas Fritz, and David Shepherd. Mining sequences of developer interactions in visual studio for usage smells. *IEEE Transactions on Software Engineering*, 43(4):359–371, 2017.
- [33] Wouter Poncin, Alexander Serebrenik, and Mark van den Brand. Process mining software development workflows. In *Proceedings of the 15th European Conference on Software Maintenance and Reengineering (CSMR)*, pages 379–382. IEEE, 2011.

- [34] Manaal Basha, Aimée M. Ribeiro, Jeena Javahar, Cleidson R. B. de Souza, and Gema Rodríguez-Pérez. CodeWatcher: IDE telemetry data extraction tool for understanding coding interactions with LLMs. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME), Tool Demonstration Track*, 2025.
- [35] Said Foutsekh, Aiko Yamashita, Fabio Petrillo, Mourad Khomh, and Yann-Gaël Guéhéneuc. Developer interaction traces backed by ide screen recordings from think-aloud sessions. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR) (Data Showcase)*. ACM, 2018. Dataset paper providing IDE interaction traces with synchronized screen recordings.
- [36] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael de Mello, Balduino Fonseca, Márcio Ribeiro, and Antonio Chávez. Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 465–475. ACM, 2017.
- [37] Stas Negara, Nicholas Chen, Mohsen Vakilian, Ralph E. Johnson, and Danny Dig. A comparative study of manual and automated refactorings. In *Proceedings of the 27th European Conference on Object-Oriented Programming (ECOOP)*, volume 7920 of *LNCS*, pages 552–576. Springer, 2013.
- [38] Mohsen Vakilian, Nicholas Chen, Stas Negara, Balaji Ambresh Rajkumar, Brian P. Bailey, and Ralph E. Johnson. Use, disuse, and misuse of automated refactorings. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, pages 233–243. IEEE, 2012.
- [39] Yida Tao and Sunghun Kim. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th IEEE/ACM Working Conference on Mining Software Repositories (MSR)*, pages 180–190. IEEE, 2015.
- [40] Marco Di Biase, Magiel Bruntink, Arie van Deursen, and Alberto Bacchelli. The effects of change decomposition on code review: A controlled experiment. *PeerJ Computer Science*, 5:e193, 2019.
- [41] Kim Herzig and Andreas Zeller. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, pages 121–130. IEEE, 2013.
- [42] Gunnar Kudrjavets, Nachiappan Nagappan, and Ayushi Rastogi. Do small code changes merge faster? A multi-language empirical investigation. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. IEEE, 2022.
- [43] G. Ann Campbell. Cognitive complexity: A new way of measuring understandability, 2018. SonarSource white paper, not peer-reviewed.
- [44] Marvin Muñoz Barón, Marvin Wyrich, and Stefan Wagner. An empirical validation of cognitive complexity as a measure of source code understandability. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2020. Best Full Paper award.
- [45] Luigi Lavazza, Abdallah Z. Abualkishik, Geng Liu, and Sandro Morasca. An empirical evaluation of the “Cognitive Complexity” measure as a predictor of code understandability. *Journal of Systems and Software*, 197:111561, 2023.
- [46] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.
- [47] Ilja Heitlager, Tobias Kuipers, and Joost Visser. A practical model for measuring maintainability. In *Proceedings of the 6th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 30–39. IEEE, 2007.
- [48] Raymond P. L. Buse and Westley R. Weimer. Learning a metric for code readability. *IEEE Transactions on Software Engineering*, 36(4):546–558, 2010.

- [49] Radu Marinescu. Detection strategies: Metrics-based rules for detecting design flaws. In *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM)*, pages 350–359. IEEE, 2004.
- [50] James M. Bieman and Byung-Kyoo Kang. Cohesion and reuse in an object-oriented system. *ACM SIGSOFT Software Engineering Notes*, 1995. Introduces Tight Class Cohesion (TCC) metric.
- [51] Stefan Wagner, Andreas Goeb, Lars Heinemann, Michael Kläs, Constanza Lampasona, Klaus Lochmann, Patrick Mäder, Reinhold Plösch, Matthias Saft, Jürgen Streit, and Adam Trendowicz. Operationalised product quality models and assessment: The Quamoco approach. *Information and Software Technology*, 2015.
- [52] Jagdish Bansiya and Carl G. Davis. A hierarchical model for object-oriented design quality assessment. *IEEE Transactions on Software Engineering*, 28(1):4–17, 2002.
- [53] Eclipse Foundation. JDT/FAQ. Eclipse Wiki, 2026. Consulted 2026-05-18.
- [54] Eclipse Foundation. Eclipse JDT Language Server (eclipse.jdt.ls). Eclipse Foundation project; reference implementation on GitHub, 2026. Newsletter article by Gorkem Ercan et al., Eclipse Newsletter, May 2017; reference implementation `eclipse-jdtls/eclipse.jdt.ls` on GitHub. Consulted 2026-05-18.
- [55] Haibo Wang, Zhuolin Xu, Huaien Zhang, Nikolaos Tsantalis, and Shin Hwei Tan. Towards understanding refactoring engine bugs. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [56] Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven LLM. In *Proceedings of the 2nd ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*. ACM, 2025.